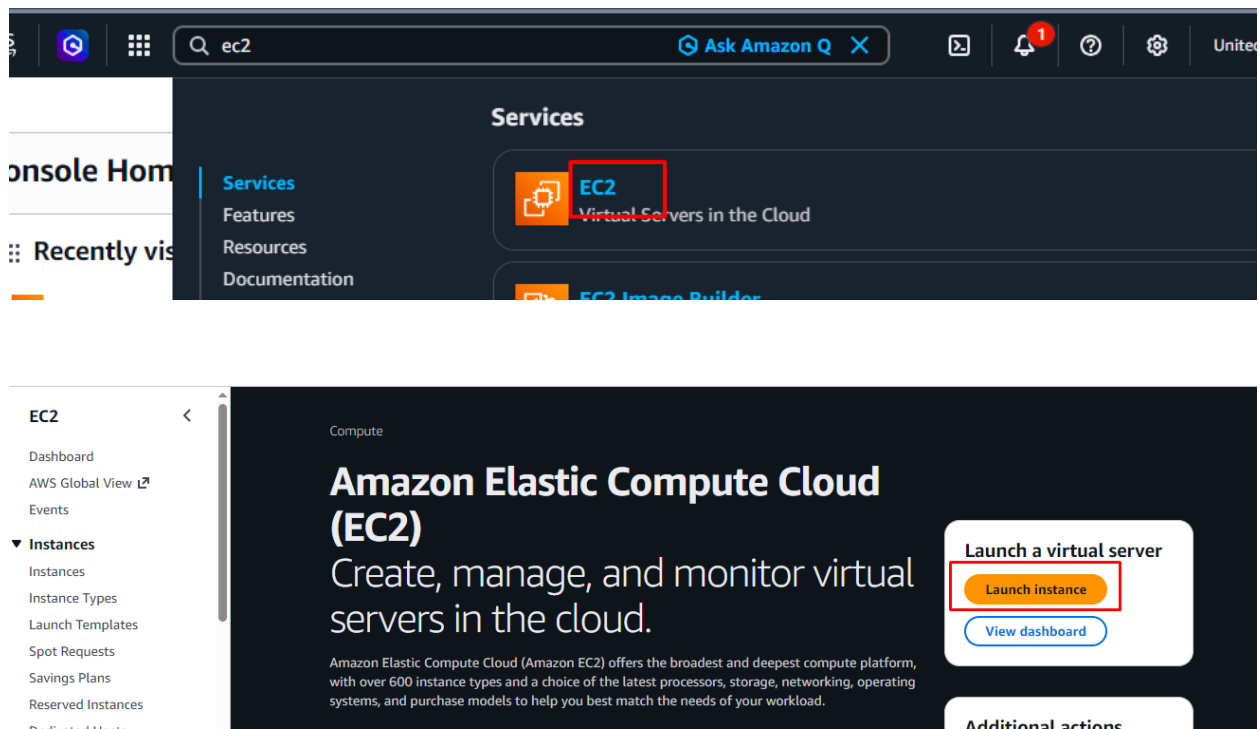


Pre-Requisites:

1. EC2 Instance – Ubuntu
2. Install Docker
3. Install aws-cli
4. Create one IAM user and provide user with ECR full access & Generate key pairs
5. Configure aws credentials in the server
6. Create one ECR repository and do the ecr login in the EC2

1.Create Ec2 instance



hroughs and continue with launching a server without any getting started guidance.

sensitive information.

Setup cost controls, secure and automatically backup your data

Configure your account settings to create budgets, require backups and data encryption. With these settings, you'll then launch a production-ready server.

🕒 5 minutes

[Get started](#)

[Cancel](#)

[Launch without a walkthrough](#)

☰ [EC2](#) > [Instances](#) > Launch an instance

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents

My AMIs

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Del

>

del

[Browse more AMIs](#)

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

<

Instances (1/1) Info

ConnectInstance state

Find Instance by attribute or tag (case-sensitive)

☒	Name	Instance ID	Instance state	Insta
☒	WORKSTATION	i-0c8a024ffc191eb8e	Running	t3.mi

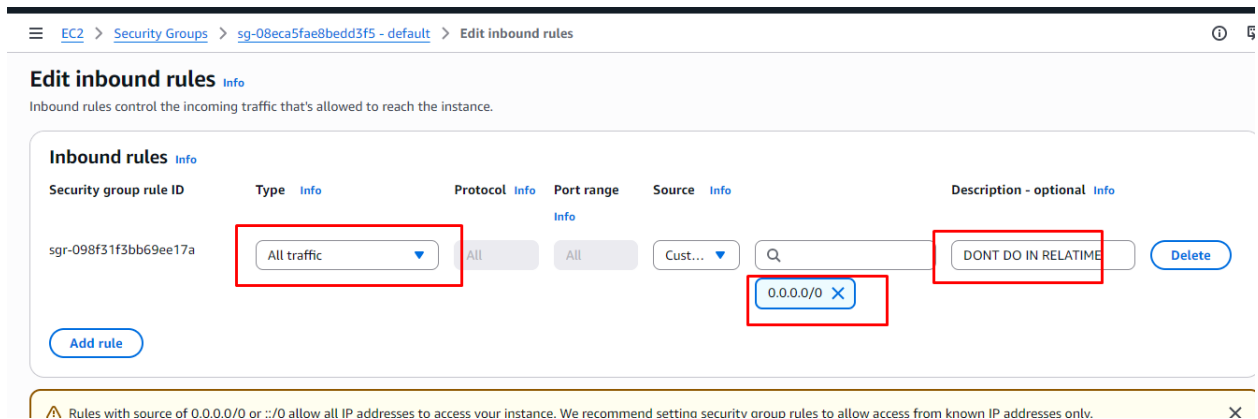
i-0c8a024ffc191eb8e (WORKSTATION)

Security groups

sg-08eca5fae8bedd3f5 (default)

Inbound rules

Outbound rules



2.Docker Installation

Login to the server and execute the below

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
sudo apt install -y ca-certificates curl gnupg lsb-release
```

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
```

```
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) \
```

```
signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
https://download.docker.com/linux/ubuntu \
```

```
$(lsb_release -cs) stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

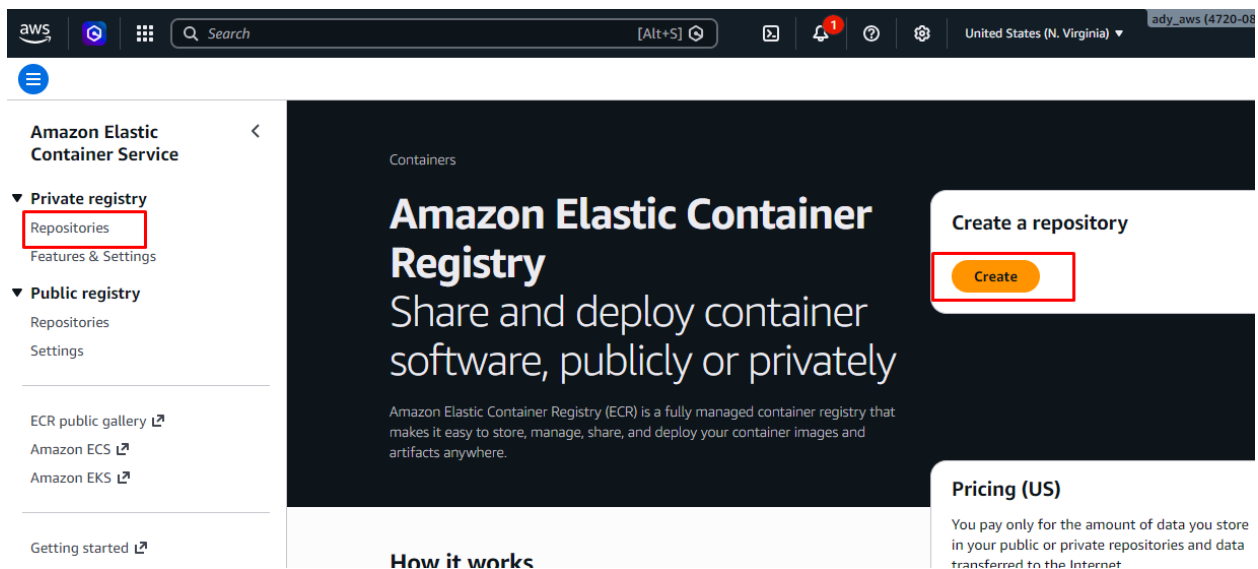
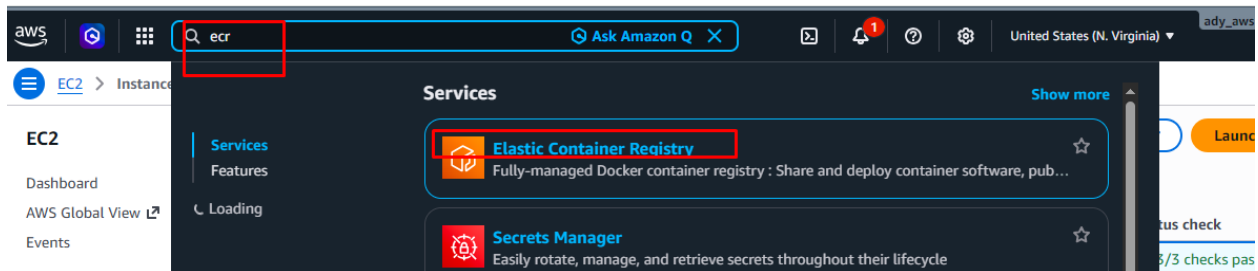
```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-  
compose-plugin
```

```
sudo docker --version
```

```
sudo docker run hello-world
```

3.Working with Registry



Create private repository

General settings

Repository name

Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.

472008046327.dkr.ecr.us-east-1.amazonaws.com/ genaiapp/app1

13 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers, and special characters _-./.

Image tag settings [Info](#)

Image tag mutability

Choose the tag mutability setting.

☒ **Mutable**
Image tags can be overwritten.

☐ **Immutable**
Image tags can't be overwritten.

Mutable tag exclusions

Tags that match these filters will be immutable (can't be overwritten). Using wildcards (*) will match zero or more image tag characters.

[Add filter](#)

Filters must only contain letters, numbers, and special characters (_ - . / *). Each filter is limited to 128 characters, 2 wildcards (*), and you can add up to 5 filters in the exclusion list.

Encryption settings [Info](#)

 The encryption settings for a repository can't be changed once the repository is created.

Encryption configuration

By default, repositories use the industry standard Advanced Encryption Standard (AES) encryption. You can optionally choose to use a key stored in the AWS Key Management Service (KMS) to encrypt the images in your repository.

☒ **AES-256**
Industry standard Advanced Encryption Standard (AES) encryption

☐ **AWS KMS**
AWS Key Management Service (KMS)

► Image scanning settings - *deprecated*

[Cancel](#)

[Create](#)

> Repositories

✔ Successfully created private repository, genaiapp/app1

Private repositories (1)



View push commands

Delete

Actions ▾

Create new

🔍 Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
☐ genaiapp/app1	472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1	March 25, 2026, 06:33:48 (UTC+04)	Mutable	AES-256

4.Pushing images to the repository

Amazon ECR > Private registry > Repositories > genaiapp/app1

Amazon Elastic Container Service

- ▼ Private registry
 - Repositories
 - [genaiapp/app1](#)
 - Features & Settings
- ▼ Public registry
 - Repositories
 - Settings

ECR public gallery [↗](#)
Amazon ECS [↗](#)
Amazon EKS [↗](#)

genaiapp/app1

Summary | **Images** | Lifecycle policy | Permissions | Repository tags

Images [Info](#) Delete Copy URI Details Scan **View push commands**

🔍 Filter active images

☐	Image tags ▾	Type	Created at ▾	Image size (MB) ▾	Image digest	Last pulled at ▾
No active images						

Push commands for genaiapp/app1



macOS / Linux

Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

1. Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 472008046327.dkr.ecr.us-east-1.amazonaws.com
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

```
docker build -t genaiapp/app1 .
```

3. After the build completes, tag your image so you can push the image to this repository:

```
docker tag genaiapp/app1:latest 472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:latest
```

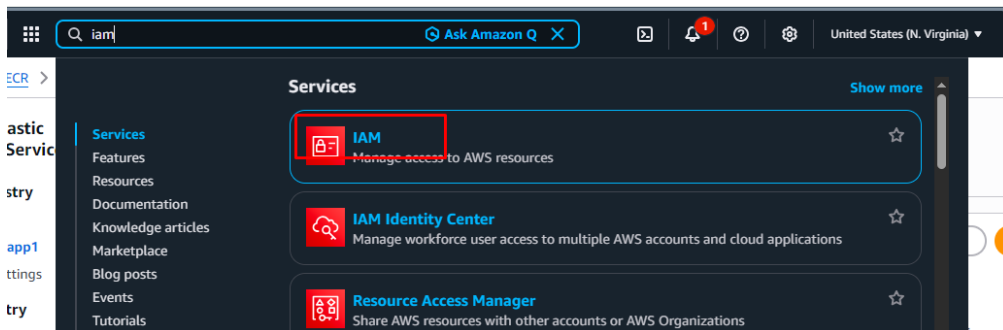
4. Run the following command to push this image to your newly created AWS repository:

```
docker push 472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:latest
```

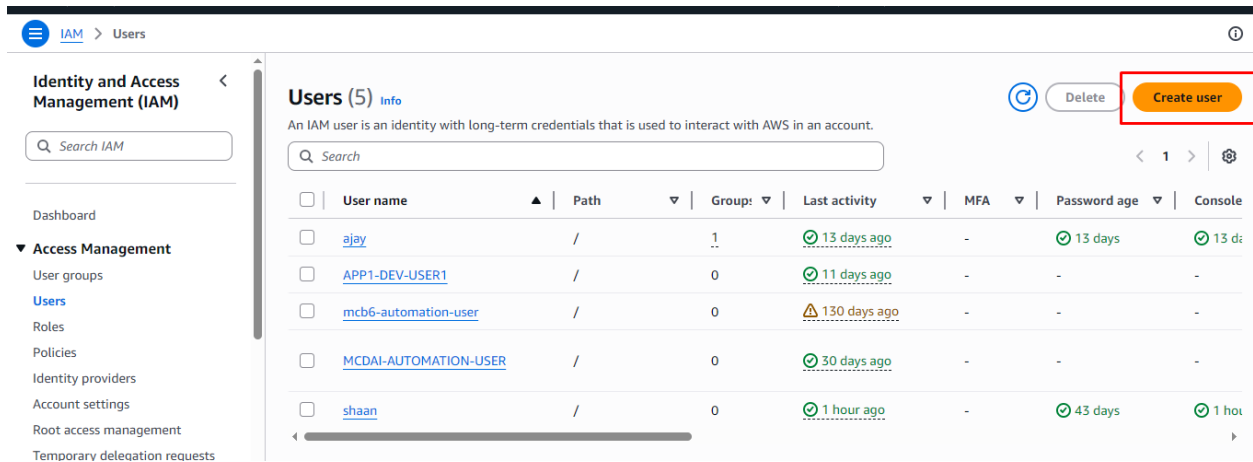
Close

5. Generate credentials to push images to ECR

IAM

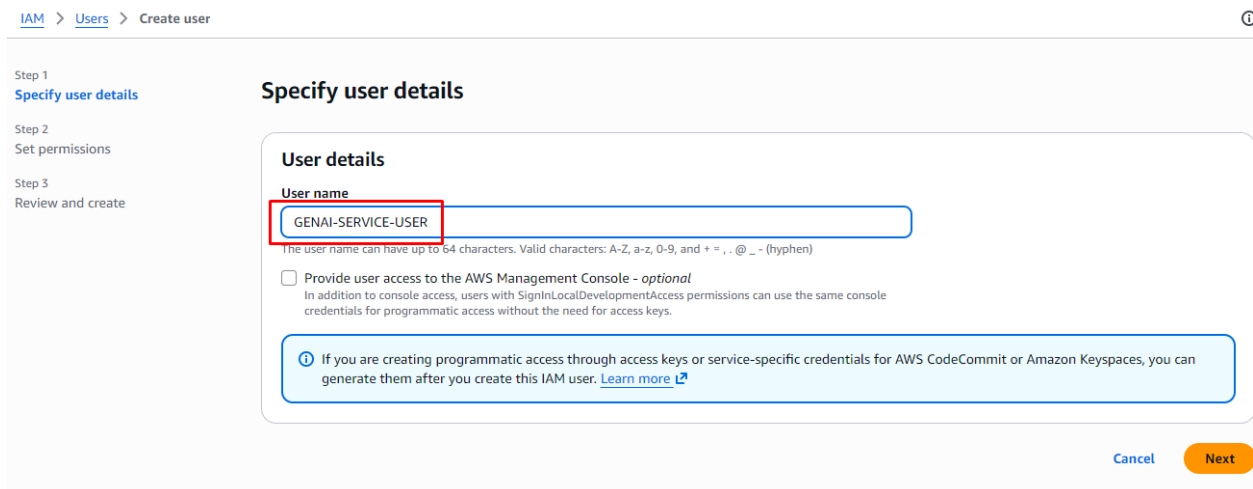


Create a service account and generate the credentials



The screenshot shows the AWS IAM console 'Users' page. On the left is a navigation menu with 'Identity and Access Management (IAM)' and 'Access Management' sections. The main area is titled 'Users (5)' and includes a search bar and a table of existing users. The 'Create user' button in the top right corner is highlighted with a red box.

☐	User name	Path	Group	Last activity	MFA	Password age	Console
☐	ajay	/	1	13 days ago	-	13 days	13 d
☐	APP1-DEV-USER1	/	0	11 days ago	-	-	-
☐	mcb6-automation-user	/	0	130 days ago	-	-	-
☐	MCDAI-AUTOMATION-USER	/	0	30 days ago	-	-	-
☐	shaan	/	0	1 hour ago	-	43 days	1 hou



The screenshot shows the 'Specify user details' step of the 'Create user' process. The 'User name' field is highlighted with a red box and contains the text 'GENAI-SERVICE-USER'. Below the field is a note about character limits and a checkbox for 'Provide user access to the AWS Management Console - optional'. A 'Next' button is visible at the bottom right.

Specify user details

User details

User name

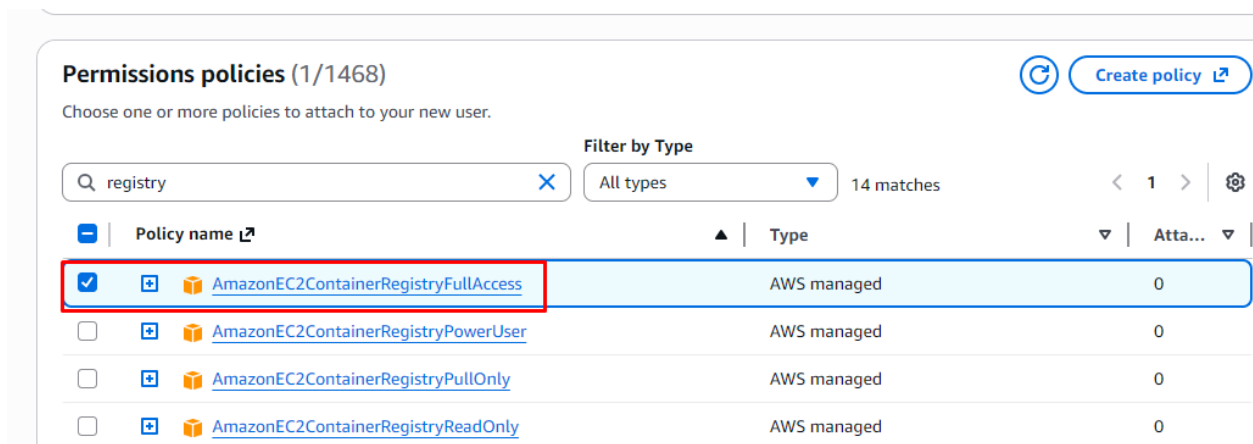
GENAI-SERVICE-USER

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☐ Provide user access to the AWS Management Console - optional

In addition to console access, users with SignInLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.

Next



The screenshot shows the 'Permissions policies' step. A search for 'registry' has been performed, resulting in 14 matches. The 'AmazonEC2ContainerRegistryFullAccess' policy is highlighted with a red box. The 'Next' button is visible at the bottom right.

Permissions policies (1/1468)

Choose one or more policies to attach to your new user.

Filter by Type

registry All types 14 matches

☐	Policy name	Type	Atta...
☒	AmazonEC2ContainerRegistryFullAccess	AWS managed	0
☐	AmazonEC2ContainerRegistryPowerUser	AWS managed	0
☐	AmazonEC2ContainerRegistryPullOnly	AWS managed	0
☐	AmazonEC2ContainerRegistryReadOnly	AWS managed	0

☐		AWSManagedReadOnlyAccess	AWS managed 0
☐		AWSServiceCatalogAppRegistryReadOnlyAccess	AWS managed 0
☐		AWSServiceCatalogAppRegistryServiceRolePolicy	AWS managed 0
☐		ROSAImageRegistryOperatorPolicy	AWS managed 0

► Set permissions boundary - optional

Cancel Previous **Next**

Permissions summary

Name ↗	Type	Used as
AmazonEC2ContainerRegistryFullAccess	AWS managed	Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

Cancel Previous **Create user**

6.Generate access keys

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- User groups
- Users**
- Roles
- Policies
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

GENAI-SERVICE-USER

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

[View user](#)

arn:aws:iam::472008046327:user/GENAI-SERVICE-USER	Console access Disabled	Access key 1 Create access key
Created March 25, 2026, 06:37 (UTC+04:00)	Last console sign-in -	

Permissions Groups Tags **Security credentials** Last Accessed

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

[Remove](#) [Add permissions](#)

Search Filter by Type All types

< 1 >

Set description tag - *optional* [Info](#)

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

Describe the purpose of this access key and where it will be used. A good description will help you rotate this access key confidently later.

Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @

[Cancel](#)

[Previous](#)

[Create access key](#)

Step 1

Access key best practices & alternatives

Step 2 - *optional*

Set description tag

Step 3

Retrieve access keys

Retrieve access keys [Info](#)

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

Secret access key

 AKIAW3ZO7G33RPTXUHWY

 ***** [Show](#)

Access key best practices

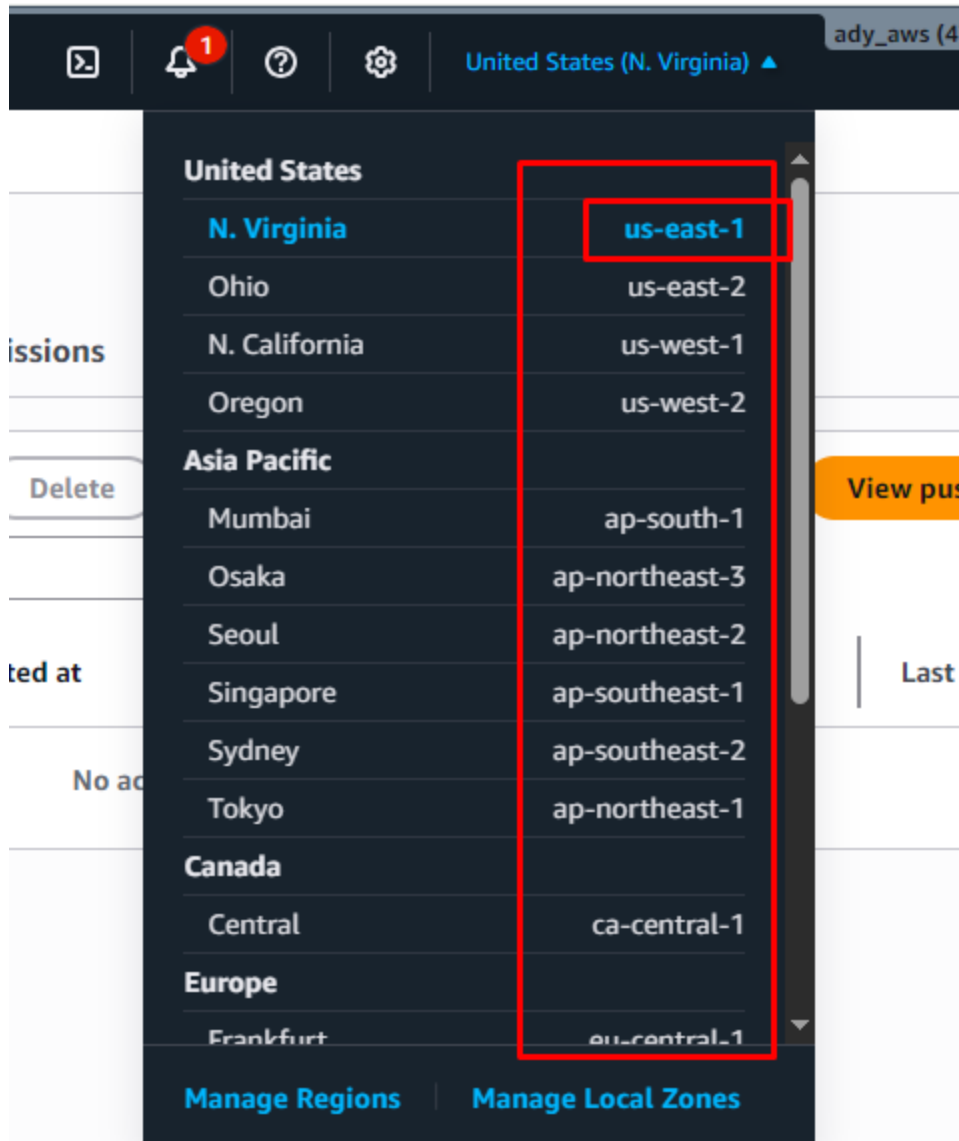
- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

7. Configure the keys in the server/laptop

Installing aws-cli software

```
snap install aws-cli --classic
```

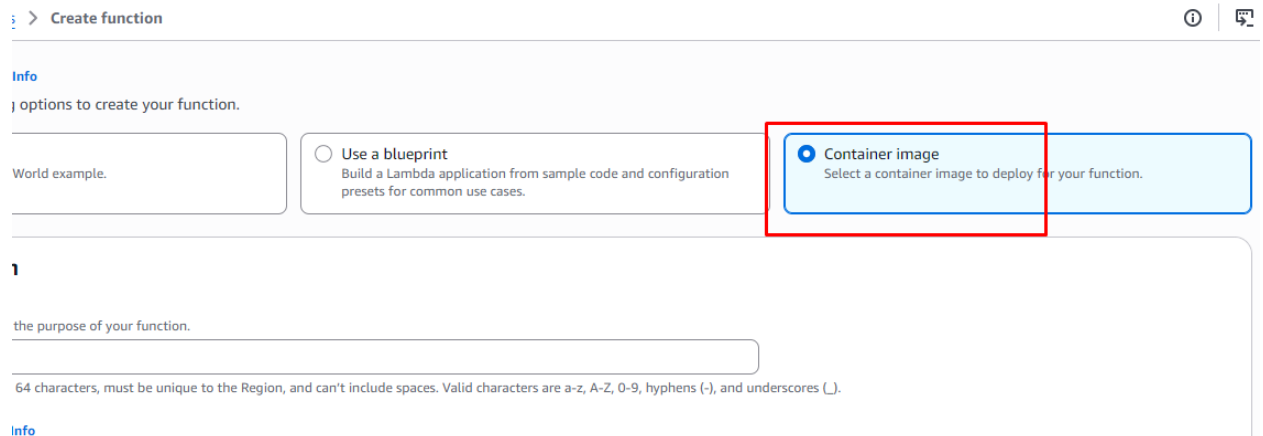
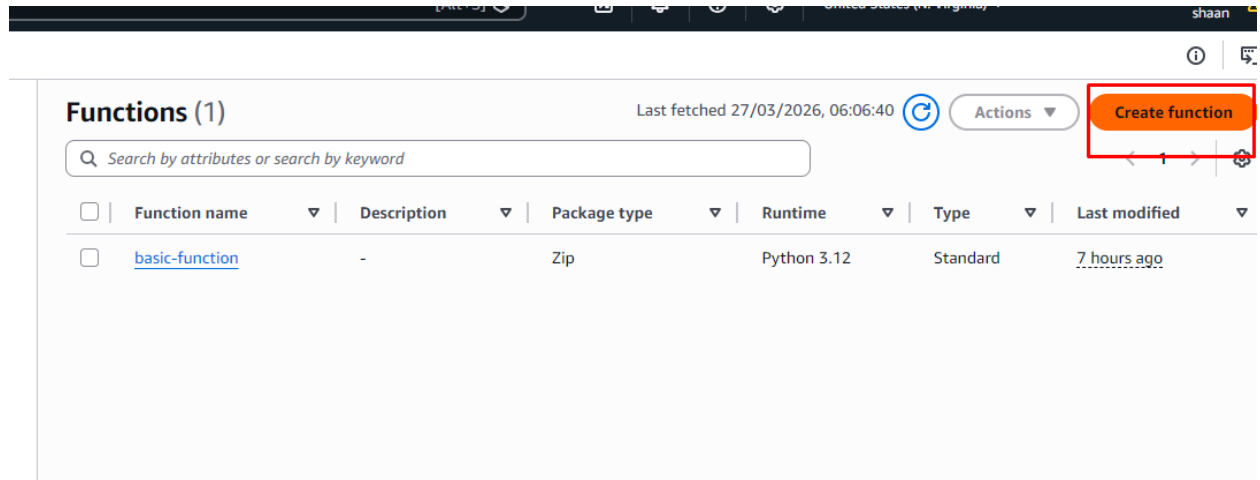
```
root@ip-172-31-92-238:~/app# snap install aws-cli --classic
error: unknown flag `classic'
root@ip-172-31-92-238:~/app# snap install aws-cli --classic
Download snap "aws-cli" (2052) from channel "v2/stable" 100% 9
```



aws configure

```
root@ip-172-31-92-238:~/app# snap install aws-cli --classic
aws-cli (v2/stable) 2.34.16 from Amazon Web Services (aws✓) installed
root@ip-172-31-92-238:~/app# aws configure
AWS Access Key ID [None]: AKIAW3ZO7G33RFP17GMY
AWS Secret Access Key [None]: QRWjpWcsRupYuDxGmpYVgxkoIM2n6G3a10J
Default region name [None]: us-east-1
Default output format [None]: json
root@ip-172-31-92-238:~/app#
```

8.Creating lambda with Container Image



Google

python + lambda + example

X | Microphone | Image Search | Search

All | Images | Videos | Short videos | Forums | Shopping | More | Tools

W3Schools

https://www.w3schools.com › python › python_lambda

Python Lambda

ExampleGet your own Python Server. Add 10 to argument a , and return the result: x = lambda a : a + 10 print(x(5)). Try it Yourself ». Lambda functions can ... [Read more](#)

DataCamp

Google

python + lambda + example

X | Microphone | Image Search | Search

Python Lambda Expressions Explained with Examples

7 Jul 2025 — A lambda expression in Python is an inline, anonymous function defined with the keyword lambda, ideal for short, one-liner operations passed as arguments. [Read more](#)

Amazon AWS Documentation

https://docs.aws.amazon.com › Developer Guide

Building Lambda functions with Python

To create a Python function · Open the Lambda console · Choose Create function · Configure the following settings: Function name: Enter a name for the function. [Read more](#)

Runtime-included SDK versions

Disabled Python features

Response format

People also search for

Python lambda example problems

Python lambda function syntax

aws

Get started | Service guides | Developer tools | AI resources

Search in this guide

Return to the Console

Invoking functions

Function scaling

Building with Node.js

Building with TypeScript

Building with Python

- Handler
- Deploy .zip file archives
- Deploy container images
- Layers
- Context
- Logging
- Testing
- Tracing

Building with Docker

Documentation > AWS Lambda > Developer Guide

Deploy Python Lambda functions with container images

PDF | RSS | Focus mode

There are three ways to build a container image for a Python Lambda function:

- Using an AWS base image for Python

The AWS base images are preloaded with a language runtime, a runtime interface client to manage the interaction between Lambda and your function code, and a runtime interface emulator for local testing.

On this page

- AWS base images for Python
- Using an AWS base image
- Using a non-AWS base image

AWS base images for Python

AWS provides the following base images for Python:

Tags	Runtime	Operating system	Dockerfile	Deprecation
3.14	Python 3.14	Amazon Linux 2023	Dockerfile for Python 3.14 on GitHub	Jun 30, 2029
3.13	Python 3.13	Amazon Linux 2023	Dockerfile for Python 3.13 on GitHub	Jun 30, 2029
3.12	Python 3.12	Amazon Linux 2023	Dockerfile for Python 3.12 on GitHub	Oct 31, 2028
3.11	Python 3.11	Amazon Linux 2	Dockerfile for Python 3.11 on GitHub	Jun 30, 2027

Using an AWS base image for Python

▼ Prerequisites

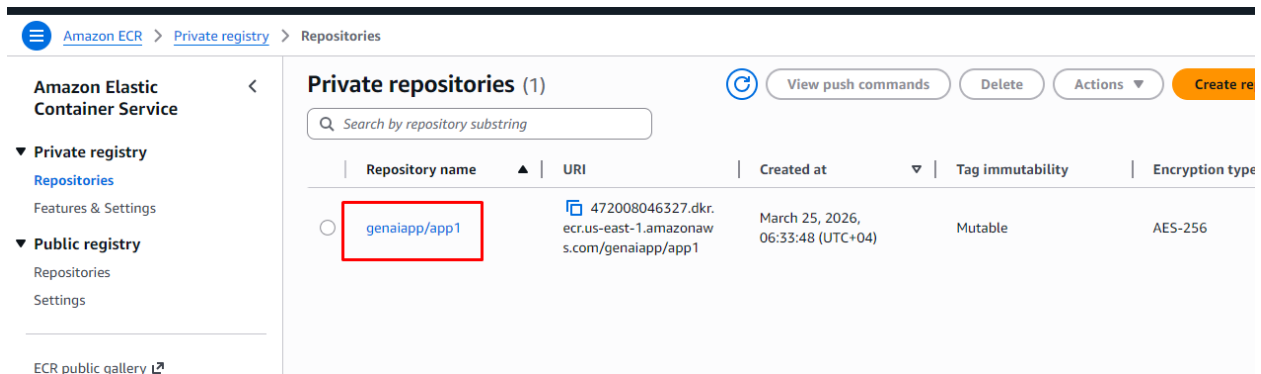
To complete the steps in this section, you must have the following:

- [AWS CLI version 2](#)
- [Docker](#) (minimum version 25.0.0)
- The Docker [buildx plugin](#).
- Python

```

root@ip-172-31-92-238:~# docker --version
Docker version 29.3.0, build 5927d80
root@ip-172-31-92-238:~# aws --version
aws-cli/2.34.16 Python/3.14.3 Linux/6.17.0-1009-aws exe/x86_64.ubuntu.24
root@ip-172-31-92-238:~# █

```



Now we have all the components required for us to start building our own image

```
mkdir /root/lambda-image
```

```
cd /root/lambda-image/
```

Application code

```
cat <<EOF >lambda_function.py
```

```
import json
```

```
def handler(event, context):
```

```
    # Sample name variable
```

```
    print(event)
```

```
    body_str = event.get("body",{})
```

```
    body = json.loads(body_str)
```

```
    print(body)
```

```
    # You can also get name dynamically from event (optional)
```

```
name = body.get('name',"DVS-TECHNOLOGIES")
message = f"Hi {name} welcome to Dvs GenAI Cloud Sessions"
# Response
return {
    "statusCode": 200,
    "body": message
}
EOF
```

```
root@ip-172-31-92-238:~/lambda-image# cat <<EOF >lambda_function.py
> import json

def handler(event, context):
    # Sample name variable
    print(event)
    body_str = event.get("body",{})
    body = json.loads(body_str)
    print(body)
    # You can also get name dynamically from event (optional)
    name = body.get('name',"DVS-TECHNOLOGIES")
    message = f"Hi {name} welcome to Dvs GenAI Cloud Sessions"
    # Response
    return {
        "statusCode": 200,
        "body": message
    }
> EOF
```

```

root@ip-172-31-92-238:~/lambda-image# cat lambda_function.py
import json

def handler(event, context):
    # Sample name variable
    print(event)
    body_str = event.get("body", {})
    body = json.loads(body_str)
    print(body)
    # You can also get name dynamically from event (optional)
    name = body.get('name', "DVS-TECHNOLOGIES")
    message = f"Hi {name} welcome to Dvs GenAI Cloud Sessions"
    # Response
    return {
        "statusCode": 200,
        "body": message
    }
root@ip-172-31-92-238:~/lambda-image# █

```

cat <<EOF >requirements.txt

boto3

EOF

```

root@ip-172-31-92-238:~/lambda-image# cat <<EOF >requirements.txt
> boto3
> EOF
root@ip-172-31-92-238:~/lambda-image# █

```

```

root@ip-172-31-92-238:~/lambda-image# ls -l
total 8
-rw-r--r-- 1 root root 427 Mar 27 02:28 lambda_function.py
-rw-r--r-- 1 root root 6 Mar 27 02:29 requirements.txt
root@ip-172-31-92-238:~/lambda-image#
root@ip-172-31-92-238:~/lambda-image# █

```

instruction is provided.

Example Dockerfile

```
FROM public.ecr.aws/lambda/python:3.12

# Copy requirements.txt
COPY requirements.txt ${LAMBDA_TASK_ROOT}

# Install the specified packages
RUN pip install -r requirements.txt

# Copy function code
COPY lambda_function.py ${LAMBDA_TASK_ROOT}

# Set the CMD to your handler (could also be done as a parameter
CMD [ "lambda_function.handler"
```

Note: whatever the name mentioned in the CMD you application file name and handler function name should match each other.

cat <<EOF > Dockerfile

```
FROM public.ecr.aws/lambda/python:3.12
```

```
# Copy requirements.txt
```

```
COPY requirements.txt \${LAMBDA_TASK_ROOT}
```

```
# Install the specified packages
```

```
RUN pip install -r requirements.txt
```

```
# Copy function code
```

```
COPY lambda_function.py \${LAMBDA_TASK_ROOT}
```

```
# Set the CMD to your handler (could also be done as a parameter override outside of the Dockerfile)
```

```
CMD [ "lambda_function.handler" ]
```

```
EOF
```

```

root@ip-172-31-92-238:~/lambda-image# cat <<EOF > Dockerfile
FROM public.ecr.aws/lambda/python:3.12

# Copy requirements.txt
COPY requirements.txt \${LAMBDA_TASK_ROOT}

# Install the specified packages
RUN pip install -r requirements.txt

# Copy function code
COPY lambda_function.py \${LAMBDA_TASK_ROOT}

# Set the CMD to your handler (could also be done as a parameter override outside of the Dockerfile)
CMD [ "lambda_function.handler" ]
EOF
root@ip-172-31-92-238:~/lambda-image# cat Dockerfile
FROM public.ecr.aws/lambda/python:3.12

# Copy requirements.txt
COPY requirements.txt \${LAMBDA_TASK_ROOT}

# Install the specified packages
RUN pip install -r requirements.txt

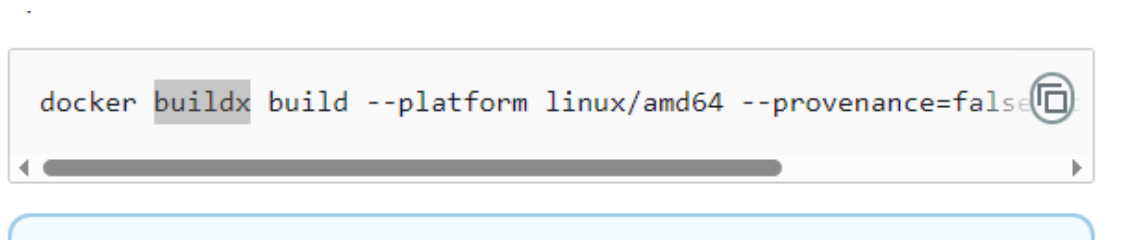
# Copy function code
COPY lambda_function.py \${LAMBDA_TASK_ROOT}

# Set the CMD to your handler (could also be done as a parameter override outside of the Dockerfile)
CMD [ "lambda_function.handler" ]
root@ip-172-31-92-238:~/lambda-image# █

```

Building docker image:

`docker buildx build --platform linux/amd64 --provenance=false -t docker-image:test .`



Now ensure you are changing the docker image name as per your ECR repository naming standards


```

root@ip-172-31-92-238:~/lambda-image# docker images

```

IMAGE	ID	DISK	USAGE	CONTENT SIZE	EX
472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:lambda	408d8aad35fa		748MB	189MB	
472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:test	39621fd7fbfa		748MB	189MB	
472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:v1.0	a61dc47d69f5	1.62GB		415MB	
docker-image:test	39621fd7fbfa		748MB	189MB	

```

root@ip-172-31-92-238:~/lambda-image#

```

Pushing image to registry

Amazon ECR > Private registry > Repositories > genaiapp/app1

Summary Images Lifecycle policy Permissions Repository tags

Images (5) Info

Filter active images

View push commands

Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
test	Image	March 25, 2026, 22:20:13 (UTC+04)	188.70	sha256:396...	March 25, 2026, 22:20:36 (UTC+04)

Login to the registry

```

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 472008046327.dkr.ecr.us-east-1.amazonaws.com

```

```

docker-image:test
root@ip-172-31-92-238:~/lambda-image# aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 472008046327.dkr.ecr.us-east-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-92-238:~/lambda-image#

```

```

docker push 472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:lambda

```

```

Login Succeeded
root@ip-172-31-92-238:~/lambda-image# docker push 472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1:lambda
The push refers to repository [472008046327.dkr.ecr.us-east-1.amazonaws.com/genaiapp/app1]
0f2edda029f6: Pushed
bf639fcde966: Layer already exists
416293068868: Pushed
db18a4b924f5: Pushed
173cad90a170: Pushed
bc9d36795909: Pushed
313f65ca458f: Pushed
0c5a64f80aa7: Pushed
d434fea271b1: Pushed
lambda: digest: sha256:408d8aad35fa6fec6fe095891a42b9ec9f3d711b87237b87c6a48d7fd66998e2 size: 2072
root@ip-172-31-92-238:~/lambda-image#

```

Amazon ECR > Private registry > Repositories > genaiapp/app1

mazon Elastic Container Service

Private registry

Repositories

genaiapp/app1

Attributes & Settings

Public registry

Amazon ECS

Amazon EKS

genaiapp/app1

Summary | **Images** | Lifecycle policy | Permissions | Repository tags

Images (6) Info [Delete](#) [Copy URI](#) [Details](#) [Scan](#) [View push commands](#)

Filter active images

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	lambda	Image	March 27, 2026, 06:44:57 (UTC+04)	188.70	sha256:408...	-
☐	test	Image	March 25, 2026, 22:20:13 (UTC+04)	188.70	sha256:396...	March 25, 2026, 22:20:36 (UTC+04)

Let's start creating our lambda function with docker image

Lambda > Functions

Successfully updated the function basic-function.

Learn to create AI applications with AWS Lambda
From agentic AI and chatbots to intelligent document processing and content generation. [Get started](#)

Functions (1) Last fetched 27/03/2026, 06:46:11 [Actions](#) [Create function](#)

Search by attributes or search by keyword

☐	Function name	Description	Package type	Runtime	Type	Last modified
☐	basic-function	-	Zip	Python 3.12	Standard	24 minutes ago

✔ Successfully updated the function **basic-function**.

Choose one of the following options to create your function.

☐ Author from scratch
Start with a simple Hello World example.

☐ Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.

☒ Container image
Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

myapp-with-docker-image

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Container image URI [Info](#)

The location of the container image to use for your function.

Enter an Amazon ECR image URI

Requires a valid Amazon ECR image URI.

Browse images

▶ Container image overrides

Select container image

Amazon ECR image repository

Select the repository containing the image that you want to use.

Select repository

Q Search Amazon ECR image repository

genaapp/app1
472008046327.dkr.ecr.us-east-1.amazonaws.com/genaapp/app1

Search images

Image tag



Digest

Select container image



Amazon ECR image repository

Select the repository containing the image that you want to use.

genaapp/app1



Images

Search Images

< 1 2 >

Image tag	Digest	Last modified
☒ lambda	sha256:408d8aad35fa6fec6fe095891a42b9ec9f3d711b87237b87c6a48d7fd66998e2	2 minutes ago
☐ test	sha256:39621fd7fbfa9d0ba49ad647d80aa8bfa1820367de6c28f8311b74e08b3edf57	1 day ago
☐	sha256:2e522834a401ad495ffc00bed69a8b9f0a010f83075c26a05b14f4c0c8a0d43	1 day ago
☐ v1.0	sha256:a61dc47d69f540aa366a0706ceb950705da8eee0ed0720607d997fc8ed07ac58	2 days ago
☐	sha256:f3f5127766fa1a38d6ac5a33259a3c7e85fea012c7dc46b1066371cf86abf2f5	2 days ago

Cancel

Select image

Successfully updated the function basic-function.

Function name

Enter a name that describes the purpose of your function.

myapp-with-docker-image

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Container image URI

The location of the container image to use for your function.

472008046327.dkr.ecr.us-east-1.amazonaws.com/genaapp/app1@sha256:408d8aad35fa6fec6fe095891a42b9ec9f3d711b87237b87c6a48d7

Browse images

Requires a valid Amazon ECR image URI

Container image overrides

Durable execution - new

Info

Container image overrides

Durable execution - new

Info Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Supports Python and Node.js runtimes. View pricing.

☐ Enable

Architecture

Info Choose the instruction set architecture you want for your function code.

☐ arm64

☒ x86_64

Permissions

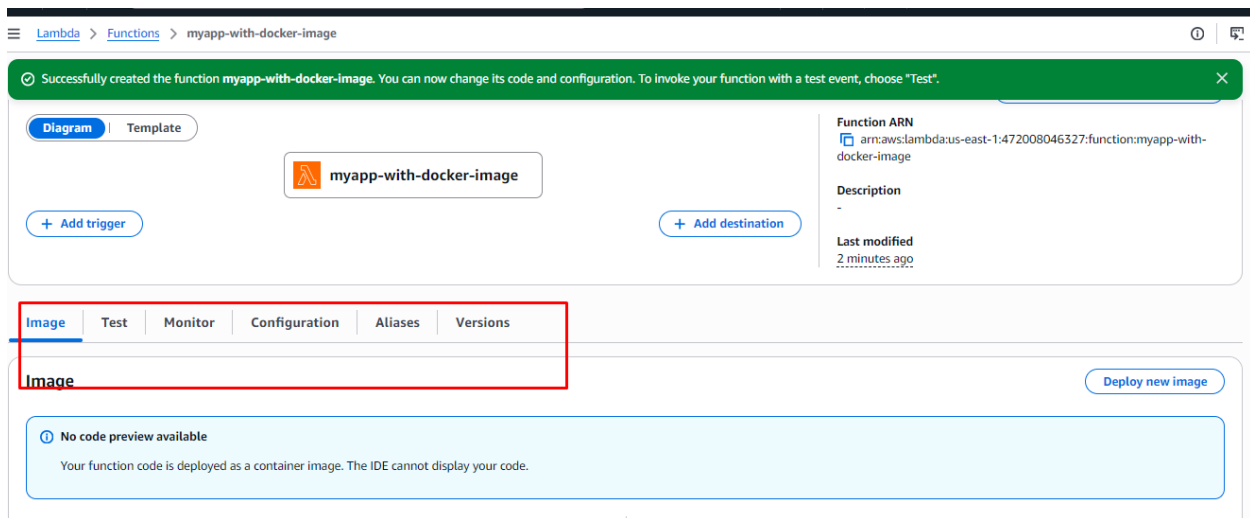
Info By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

Change default execution role

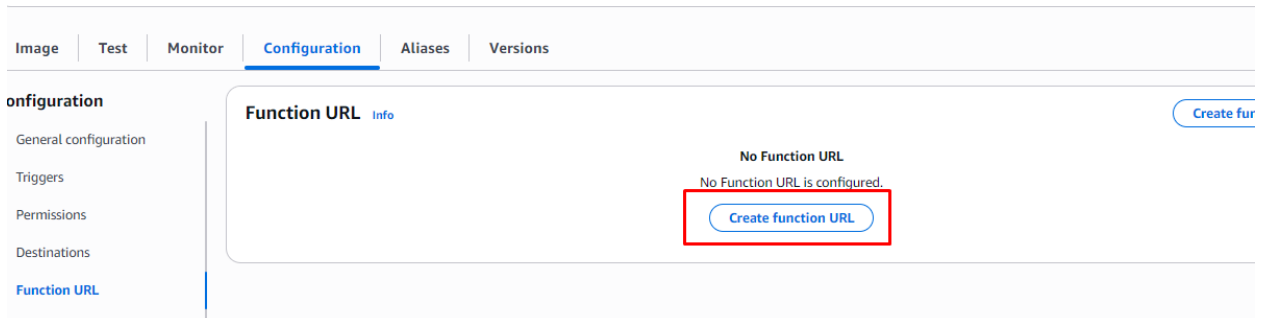
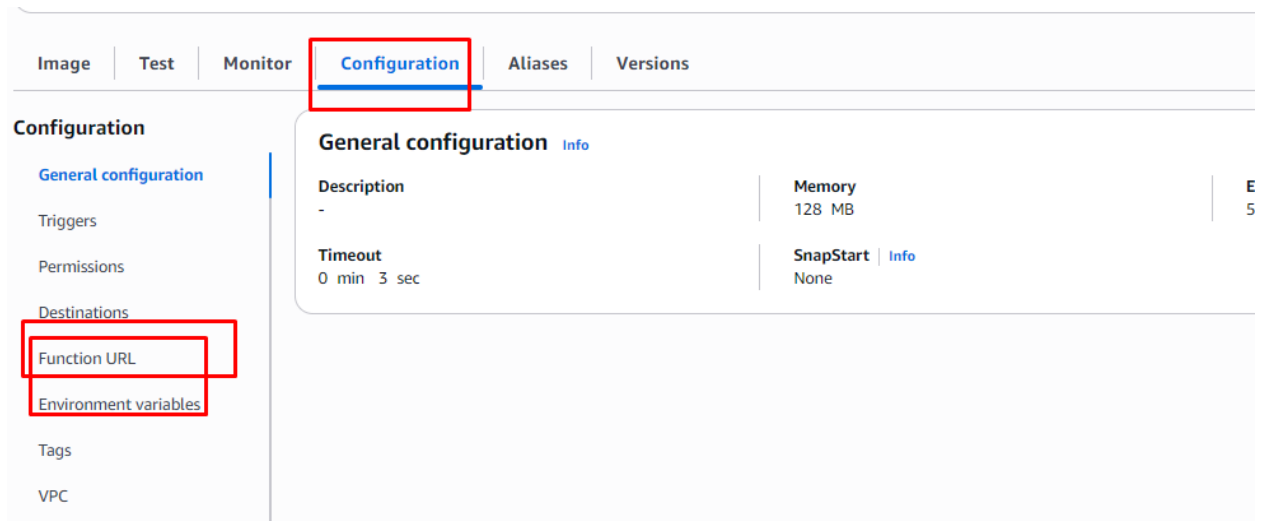
Additional configurations

Cancel

Create function



Let's expose our lambda function with url



Function URL [Info](#)

Use function URLs to assign HTTP(S) endpoints to your Lambda func

Auth type

Choose the auth type for your function URL. [Learn more](#) [↗](#)

☐ AWS_IAM

Only authenticated IAM users and roles can make requests to your function

☒ NONE

Lambda won't perform IAM authentication on requests to your function UR

Function URL permissions

Cancel Save

Cancel

Save

✔ Your changes have been saved.

Image Test Monitor **Configuration** Aliases Versions

Configuration

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

VPC

Function URL [Info](#)

ⓘ Your function URL is public. Anyone with the URL can access your function.

Function URL



<https://cljkem2r366pqemdcu6h7shrue0qnmzi.lambda-url.us-east-1.on.aws/> [↗](#)

Creation time

2 seconds ago

CORS (Not enabled)

Auth type

NONE

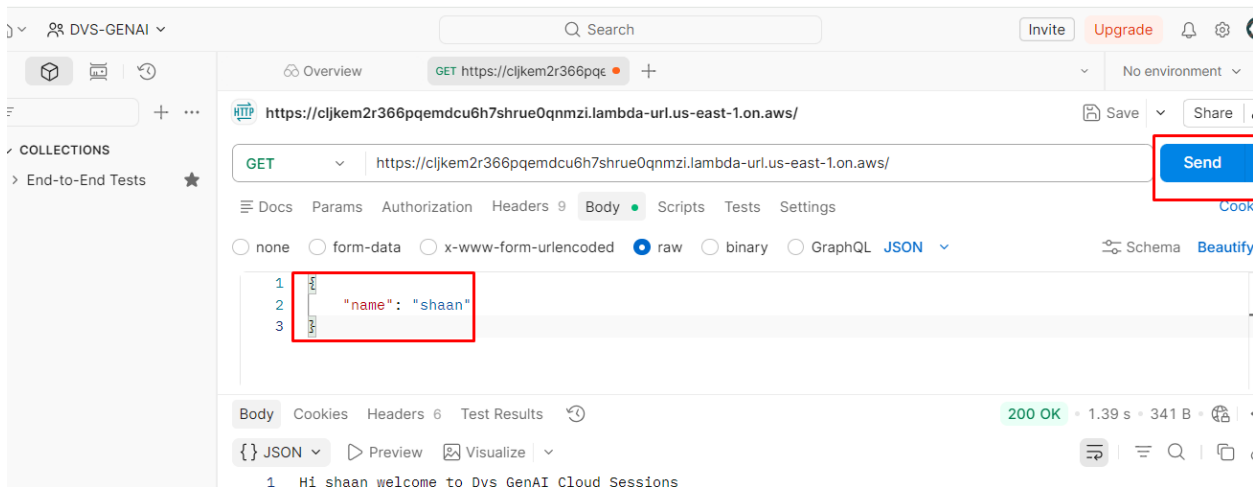
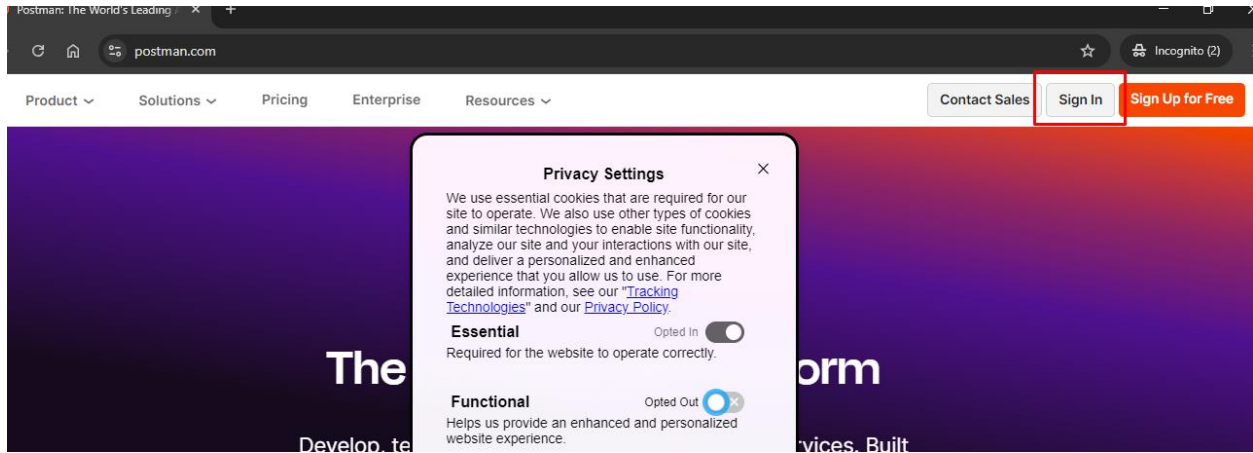
Last modified

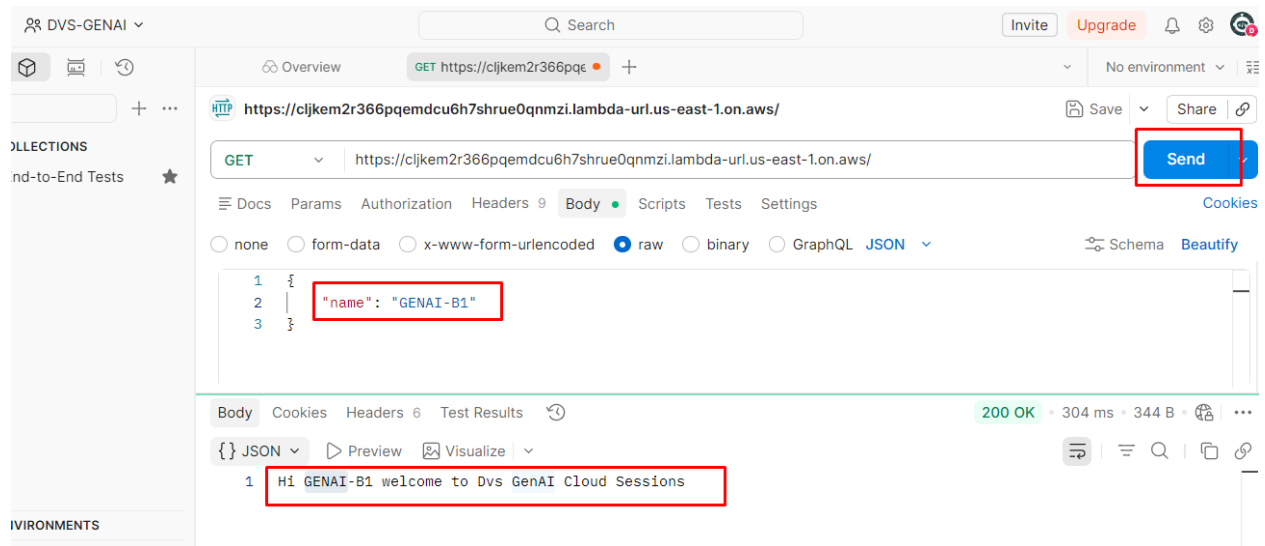
2 seconds ago

Final testing

Login to postman and then test the application using below

<https://www.postman.com/>





Finally we can see that our application is running with our image in the lambda platform